



《AI大模型Ragent项目》——知识库文档开始分块接口

来自： 拿个offer-开源&项目实战



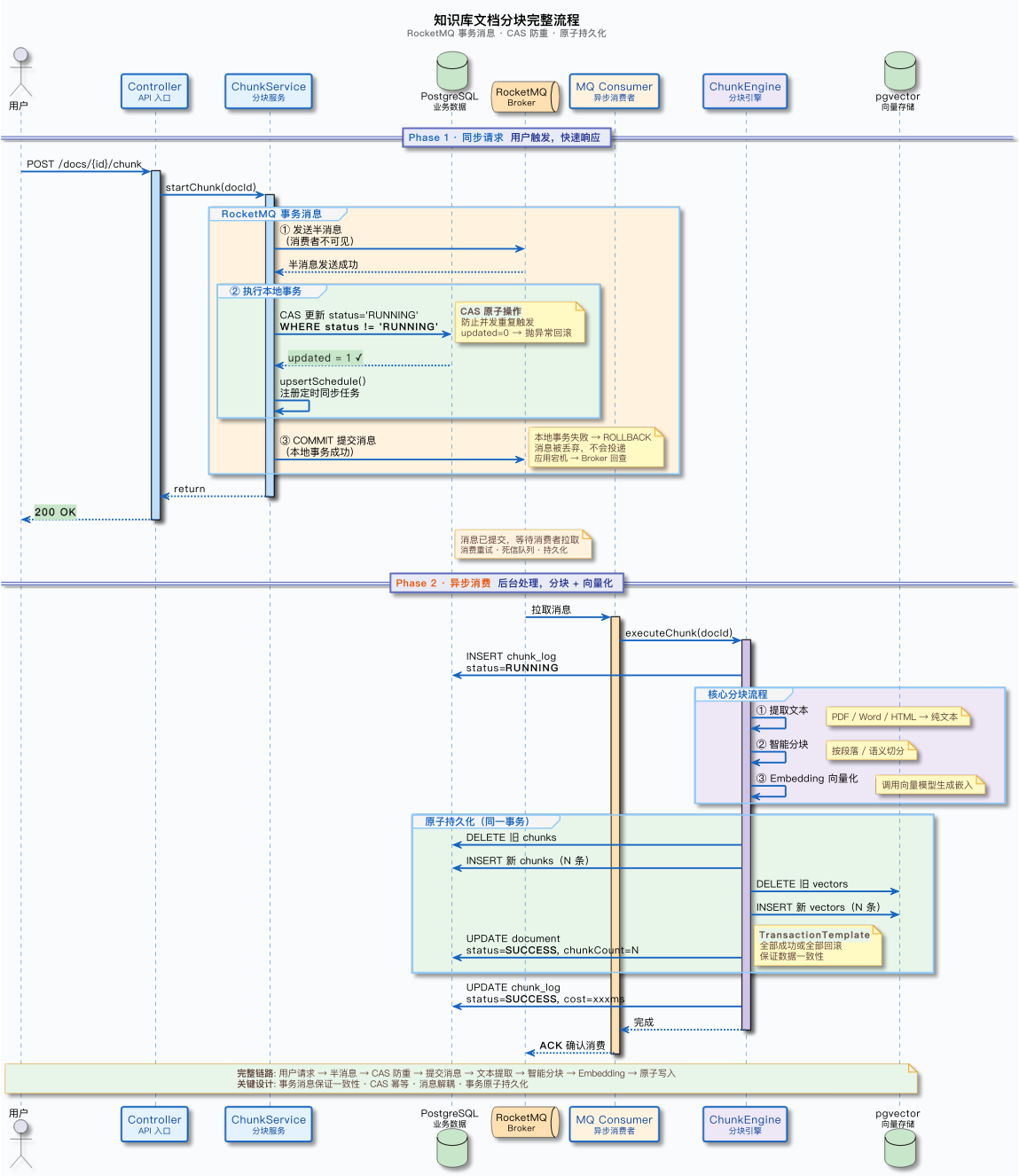
马丁

2026年03月29日 12:48

前面一篇文章把文件上传的完整流程讲完了：用户上传文件，系统把文件存到对象存储，把文档元信息存到数据库，状态设为 pending。
upload 接口不做分块、不做向量化，这些耗时操作由后续的分块接口异步触发。
这篇文章就来讲分块是怎么触发的，以及从触发到完成的完整异步处理链路。

完整流程概览

在看具体代码之前，先用一张时序图建立全局视角。从用户点击执行分块按钮，到分块完成，中间经过了哪些环节：



整个流程可以分为两个阶段：

1. **同步阶段：** 用户点击执行分块 → Controller 调用 startChunk → 通过 RocketMQ 事务消息保证 CAS 状态更新和消息发送的原子性 → 立即返回。这个阶段只涉及数据库更新和消息发送，响应很快。
2. **异步阶段：** MQ Consumer 消费消息 → 执行分块任务 → 原子性写入数据库和向量库 → 更新状态。这个阶段包含文件解析、分块、向量化等耗时操作，处理时间较长。

用户不需要等异步阶段完成，点击执行分块后立即返回，可以继续做其他事情。前端可以通过轮询接口获取文档状态，展示处理进度。

为什么拆成两个接口

你可能会问：为什么不在 upload 接口中自动触发分块，而要单独做一个 startChunk 接口？

1. 上传和分块的职责差异

upload 接口做的事情：把文件存到对象存储，把元数据存到数据库。这个过程虽然涉及网络 IO，但对于单个文件来说，耗时是可控的。
startChunk 接口做的事情：解析文件提取文本、分块、调用 Embedding API 生成向量、写入向量库。这个过程耗时长得多。一个较大的 PDF 文档，需要经过文件解析、文本分块、多次 Embedding API 调用等环节，整个流程可能需要较长时间。
如果把这两个操作合在一起，用户上传文件后需要等待很长时间才能看到结果，体验很差。

2. 拆分的好处

拆成两个接口有这些好处：

职责清晰：upload 只管存文件，startChunk 只管分块。每个接口做一件事，代码更好维护。

灵活控制：用户可以上传多个文档后，批量触发分块。也可以文档更新后，只重新分块不重新上传文件。

支持重试：分块失败后，用户可以直接重新触发分块，不需要重新上传文件。

方便扩展：后续如果要支持分块参数调整（比如改变 chunkSize），只需要重新触发分块，不需要重新上传文件。

startChunk 方法详解

1. 接口入口

Controller 层的代码很简单：

```
@PostMapping("/knowledge-base/docs/{doc-id}/chunk")
public Result<Void> startChunk(@PathVariable("doc-id") String docId) {
    documentService.startChunk(docId);
    return Results.success();
}
```

接口路径 /knowledge-base/docs/{doc-id}/chunk，只需要一个路径参数 docId，不需要请求体。调用 Service 层的 startChunk 方法，立即返回，不等处理完成。

2. startChunk 核心逻辑

来看 KnowledgeDocumentServiceImpl#startChunk 方法的完整实现：

```
@Override
public void startChunk(String docId) {
    KnowledgeDocumentChunkEvent event = KnowledgeDocumentChunkEvent.builder()
        .docId(docId)
        .operator(UserContext.getUsername())
        .build();

    // 发送 RocketMQ 事务消息，本地事务和消息发送保证原子性
    messageQueueProducer.sendInTransaction(
        chunkTopic,
        docId,
        "文档分块",
        event,
        arg -> {
            // ---- 本地事务逻辑 ----
            // 原子 CAS 更新：WHERE status != 'running', 并发请求只有一个能成功
            int updated = documentMapper.update(
                new LambdaUpdateWrapper<KnowledgeDocumentDO>()
                    .set(KnowledgeDocumentDO::getStatus, DocumentStatus.RUNNING.getStatusCode())
                    .set(KnowledgeDocumentDO::getUpdatedBy, event.getOperator())
                    .eq(KnowledgeDocumentDO::getId, docId)
                    .ne(KnowledgeDocumentDO::getStatus, DocumentStatus.RUNNING.getStatusCode())
            );
        }
    );
}
```

```
        if (updated == 0) {
            KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
            Assert.notNull(documentDO, () -> new ClientException("文档不存在"));
            throw new ClientException("文档分块操作正在进行中，请稍后再试");
        }

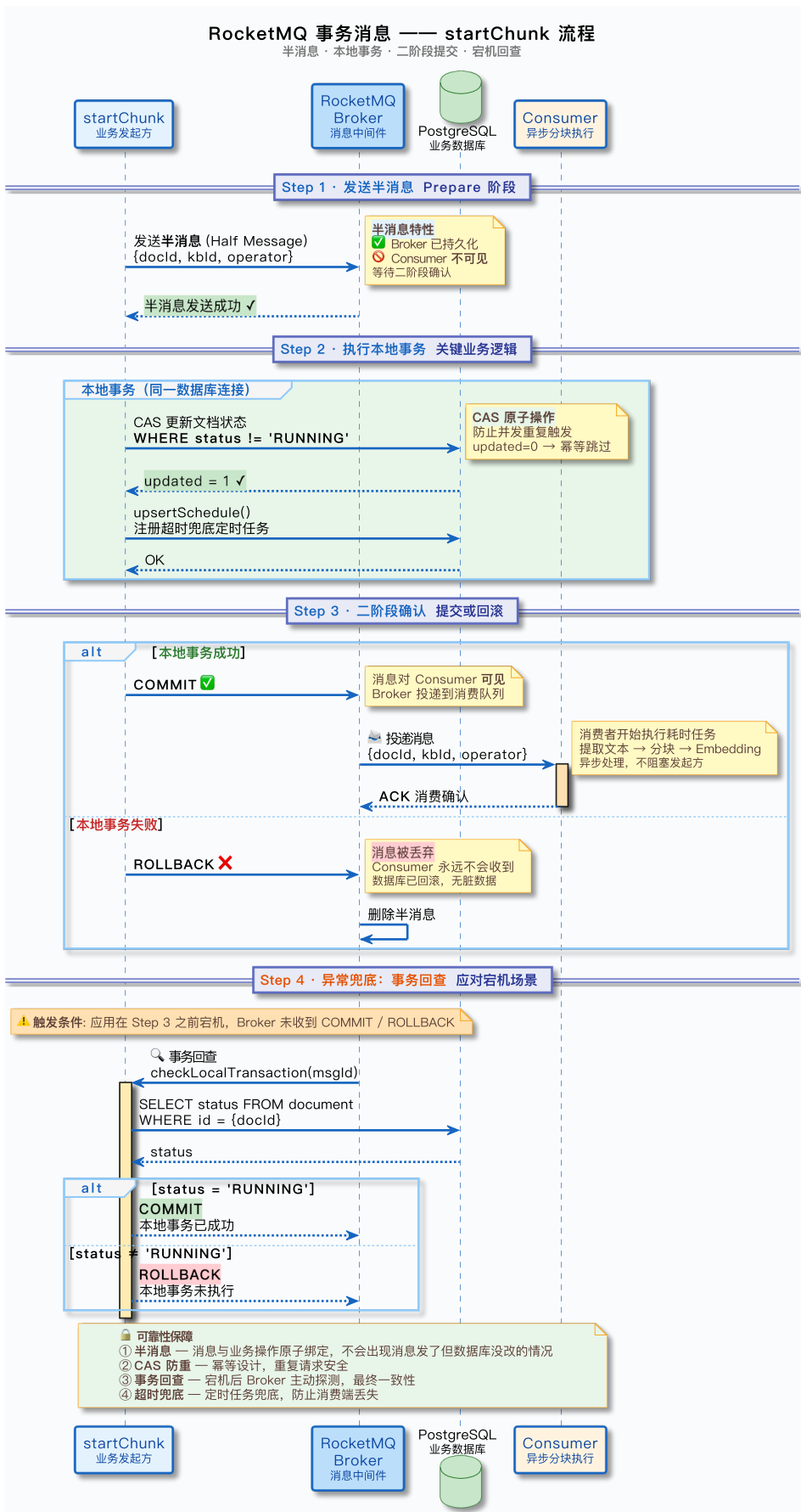
        KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
        event.setKbId(documentDO.getKbId());
        scheduleService.upsertSchedule(documentDO);
    }

    });
}
```

和之前版本最大的区别：方法不再用 `@Transactional` 注解，而是通过 RocketMQ 事务消息把本地事务和消息发送绑定在一起。CAS 状态更新、定时任务注册这些数据库操作，作为本地事务逻辑传入 `sendInTransaction` 方法的回调中。

整个方法只做了一件事：发送一条 RocketMQ 事务消息，本地事务回调中完成 CAS 更新状态和注册定时任务。先用一张时序图看完整流程，再逐个拆解各环节。





下面逐个拆解各环节的细节。

2.1 原子 CAS 状态更新

第一步是更新文档状态, 从非 running 改为 running。这里用的是 CAS (Compare-And-Swap) 操作, 通过 MyBatis Plus 的 ne(status, RUNNING) 条件实现。

SQL 翻译过来是这样的:

```
UPDATE t_knowledge_document
SET status = 'running', updated_by = 'xxx'
```

```
WHERE id = ? AND status != 'running'
```

关键在于 WHERE status != 'running' 这个条件。如果文档当前状态已经是 running，这条 UPDATE 不会匹配到任何记录，updated 返回 0。

为什么需要这个条件？因为用户可能手抖点了两次执行分块按钮，或者多个用户同时点击，或者前端因为网络问题重复发送了请求。如果没有这个条件，两个请求都会成功，会触发两次分块任务，导致重复处理。

有了这个条件，第一个请求把状态改成 running，第二个请求发现状态已经是 running，UPDATE 不会匹配到记录，updated 返回 0，方法抛出异常告诉用户文档正在处理中。

这就是 CAS 的核心思想：只有当前值符合预期时才更新，否则更新失败。这是一种乐观并发控制机制，不需要加锁，性能很好。

2.2 定时任务注册

第二步是调用 scheduleService.upsertSchedule(documentD0)，为 URL 来源的文档创建或更新定时刷新任务。

还记得 upload 接口中讲的定时同步功能吗？用户上传 URL 类型的文档时，可以设置 scheduleEnabled=true 和 scheduleCron 表达式，系统会按照 cron 定义的周期自动重新抓取远程文件并触发分块更新。

upsertSchedule 方法会检查文档的来源类型和定时配置，如果满足条件（来源是 URL、定时开关打开、cron 表达式合法），就在 t_knowledge_document_schedule 表中创建或更新一条定时任务记录。

定时任务的底层调度机制（怎么扫描、怎么触发、怎么加锁）比较复杂，后续文章会专门讲。这里只需要知道：startChunk 方法会确保定时任务配置和文档配置保持同步。

2.3 发送事务消息

第三步是发送 RocketMQ 事务消息。注意看代码结构：CAS 更新和定时任务注册都在 sendInTransaction 的回调 Lambda 里面，而不是在外层方法里。

这意味着：如果本地事务（CAS 更新）执行成功，消息才会被投递给消费者；如果本地事务失败（比如 CAS 返回 0 抛出异常），消息会被丢弃，不会有消费者收到这条消息。

消息体是 KnowledgeDocumentChunkEvent，包含三个字段：

```
@Data
@Builder
public class KnowledgeDocumentChunkEvent {
    private String docId;    // 文档 ID
    private String kbId;     // 知识库 ID
    private String operator; // 操作人（用于审计）
}
```

注意 kbId 不是在构建 event 时设置的，而是在本地事务回调中查询文档后通过 event.setKbId(documentD0.getKbId()) 设置的。因为查询文档是在本地事务内部，这时候才能拿到完整的文档信息。

2.4 为什么用事务消息

前面讲的 CAS 更新和消息发送，看起来可以用更简单的方式实现：先更新数据库，再发消息。但这样有一个经典的分布式问题：**数据库更新成功了，消息发送失败了怎么办？**

数据库里文档状态已经变成了 running

但消息没发出去，不会有消费者来执行分块任务

文档就卡在 running 状态，用户无法重新触发，也等不到结果

反过来也有问题：**消息发送成功了，数据库更新失败了怎么办？**

消费者收到消息开始分块

但数据库里文档状态还是 pending，数据不一致

RocketMQ 事务消息就是为了解决这个问题。它的核心思想是：把本地数据库操作和消息发送绑定成一个原子操作，要么都成功，要么都失败。

简单说一下事务消息的工作流程：

1. **发送半消息**：先把消息发到 RocketMQ Broker，但标记为半消息，消费者看不到
2. **执行本地事务**：执行回调中的数据库操作（CAS 更新状态、注册定时任务）
3. **提交或回滚**：本地事务成功 → 通知 Broker 提交消息，消费者可以消费；本地事务失败 → 通知 Broker 回滚消息，消息被丢弃
4. **事务回查**：如果 Broker 没收到提交/回滚通知（比如应用宕机了），Broker 会主动回查本地事务状态

这样就保证了：数据库状态更新和消息投递要么同时成功，要么同时失败。

事务消息的内部实现细节（半消息存储、回查机制、超时处理等）比较复杂，后续会有单独的文章专门讲解。这里只需要知道它解决了什么问题、怎么使用就够了。

2.5 事务回查

如果应用在执行完本地事务后、通知 Broker 之前宕机了，Broker 不知道这条半消息该提交还是回滚。这时候 Broker 会主动回查。

项目中注册了一个事务回查器 `KnowledgeDocumentChunkTransactionChecker`：

```
@Override
public boolean check(MessageWrapper<?> message) {
    KnowledgeDocumentChunkEvent event = (KnowledgeDocumentChunkEvent) message.getBody();
    String docId = event.getDocId();
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    // 如果文档状态已经是 running，说明本地事务已提交，消息应该投递
    return documentDO != null
        && DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus());
}
```

回查逻辑很简单：查数据库看文档状态是不是 `running`。如果是，说明本地事务已经提交成功了，消息应该投递；如果不是，说明本地事务没有执行或者回滚了，消息应该丢弃。

异步处理链路

`startChunk` 方法发送 MQ 消息后立即返回，真正的分块处理在异步线程中执行。来看异步处理的完整链路。

1. MQ Consumer 消费消息

`KnowledgeDocumentChunkConsumer` 是 MQ 消息的消费者，通过 `@RocketMQMessageListener` 注解注册：

```
@Component
@RequiredArgsConstructor
@RocketMQMessageListener(
    topic = "knowledge-document-chunk_topic${unique-name:}",
    consumerGroup = "knowledge-document-chunk_cg${unique-name:}"
)
public class KnowledgeDocumentChunkConsumer implements RocketMQListener<MessageWrapper<KnowledgeDo

    private final KnowledgeDocumentService documentService;

    @Override
    public void onMessage(MessageWrapper<KnowledgeDocumentChunkEvent> message) {
        KnowledgeDocumentChunkEvent event = message.getBody();
        log.info("[消费者] 开始消费文档分块任务, docId={}, keys={},", event.getDocId(), message.getKeys(

        // 设置 UserContext，因为消费者线程没有 HTTP 上下文
        UserContext.set(LoginUser.builder().username(event.getOperator()).build());
        try {
            documentService.executeChunk(event.getDocId());
        } finally {
            UserContext.clear();
        }
    }
}
```

消费者做了两件事：

1. **设置 UserContext**：消费者线程是异步线程，没有 HTTP 请求上下文，需要手动设置 UserContext。operator 字段就是为了这个目的，记录是谁触发的分块操作，方便审计。
2. **调用 executeChunk**：执行实际的分块逻辑。

2. executeChunk 方法

executeChunk 方法是一个简单的包装：

```
@Override
public void executeChunk(String docId) {
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    if (documentDO == null) {
        log.warn("文档不存在，跳过分块任务，docId={}", docId);
        return;
    }
    runChunkTask(documentDO);
}
```

查询文档记录，如果文档不存在（可能被删除了），直接返回。否则调用 runChunkTask 执行分块任务。

3. runChunkTask 核心流程

runChunkTask 是分块处理的核心方法，包含了完整的处理流程和异常处理。方法比较长，我们分段来看。

3.1 创建分块日志

方法的第一步是创建分块日志记录：

```
KnowledgeDocumentChunkLogDO chunkLog = KnowledgeDocumentChunkLogDO.builder()
    .docId(docId)
    .status(DocumentStatus.RUNNING.getCode())
    .processMode(processMode.getValue())
    .chunkStrategy(documentDO.getChunkStrategy())
    .pipelineId(documentDO.getPipelineId())
    .startTime(new Date())
    .build();
chunkLogMapper.insert(chunkLog);
```

分块日志记录在 t_knowledge_document_chunk_log 表中，每次执行分块任务都会创建一条新记录。这个表的作用是记录分块任务的执行情况，包括处理模式、分块策略、各阶段耗时、成功或失败等信息。

为什么要单独记录日志，而不是直接更新文档表的状态？因为文档可能会多次重新分块（比如调整分块参数后重新处理），每次执行的情况都不一样。单独的日志表可以保留完整的历史记录，方便排查问题和性能分析。

3.2 处理模式分发

第二步是根据文档的处理模式分发到不同的处理逻辑：

```
List<VectorChunk> chunkResults;
if (ProcessMode.PIPELINE == processMode) {
    long start = System.currentTimeMillis();
    chunkResults = runPipelineProcess(documentDO);
    chunkDuration = System.currentTimeMillis() - start;
} else {
    ChunkProcessResult result = runChunkProcess(documentDO);
    extractDuration = result.extractDuration();
    chunkDuration = result.chunkDuration();
    embedDuration = result.embedDuration();
    chunkResults = result.chunks();
}
```

项目支持两种处理模式：

CHUNK 模式：调用 runChunkProcess 方法，执行固定的处理流程：

1. 从对象存储读取文件
2. 用 Apache Tika 解析文件，提取纯文本
3. 根据文档配置的分块策略（chunkStrategy）和分块参数（chunkConfig）执行分块

4. 调用 Embedding API 为每个 chunk 生成向量
5. 返回 List<VectorChunk>, 每个 VectorChunk 包含文本内容、向量、元数据

这是最常用的模式，适合大部分文档。

PIPELINE 模式：调用 runPipelineProcess 方法，执行用户自定义的 Pipeline 流程：

1. 读取文件内容
2. 构建 IngestionContext, 设置 skipIndexerWrite=true (关键参数, 后面会讲)
3. 执行 Pipeline 定义的节点序列 (可能包括: 获取节点 → 解析节点 → 清洗节点 → 分块节点 → 增强节点等)
4. 从 context 中提取 chunks
5. 返回 List<VectorChunk>

PIPELINE 模式更灵活, 允许用户自定义处理流程。比如在分块之前加一个文本清洗节点, 去掉特殊字符; 或者在分块之后加一个增强节点, 为每个 chunk 生成摘要。

PIPELINE 模式的详细设计 (节点定义、编排方式、执行引擎) 比较复杂, 后续文章会专门讲。这里只需要知道: 两种模式最终都返回 List<VectorChunk>, 后续的处理流程是一样的。

有一个关键细节: PIPELINE 模式中设置了 skipIndexerWrite=true。这是什么意思?

Pipeline 中有一个 IndexerNode (索引节点), 负责把 chunks 写入向量库。但如果让 IndexerNode 直接写入, 就无法和数据库的 chunk 表写入保持原子性 (一个成功另一个失败)。所以这里设置 skipIndexerWrite=true, 让 IndexerNode 跳过写入操作, 只是把 chunks 放到 context 中。真正的写入在后面的 persistChunksAndVectorsAtomically 方法中, 和数据库写入放在同一个事务里, 保证原子性。

3.3 原子性写入

第三步是把 chunks 和 vectors 原子性地写入数据库和向量库:

```
long persistStart = System.currentTimeMillis();
String collectionName = resolveCollectionName(documentD0.getKbId());
int savedCount = persistChunksAndVectorsAtomically(collectionName, docId, chunkResults);
persistDuration = System.currentTimeMillis() - persistStart;
```

这是整个流程中最关键的一步, 后面会单独用一个大节详细讲。这里先知道: 这个方法在一个事务中完成了五个操作, 要么全部成功, 要么全部失败。

3.4 更新分块日志

第四步是更新分块日志, 记录成功状态和各阶段耗时:

```
long totalDuration = System.currentTimeMillis() - totalStartTime;
updateChunkLog(chunkLog.getId(), DocumentStatus.SUCCESS.getCode(), savedCount,
    extractDuration, chunkDuration, embedDuration, persistDuration, totalDuration, null);
```

updateChunkLog 方法会更新日志记录的这些字段:

```
status: success
chunkCount: 生成的 chunk 数量
extractDuration: 文本提取耗时 (毫秒)
chunkDuration: 分块耗时 (毫秒)
embedDuration: 嵌入 API 调用耗时 (毫秒)
persistDuration: DB 持久化耗时 (毫秒)
totalDuration: 总耗时 (毫秒)
endTime: 结束时间
```

这些指标对性能分析很有用。比如发现某个文档的 extractDuration 特别长, 说明文件解析慢, 可能是 PDF 格式复杂或者文件很大。如果 embedDuration 很长, 说明 Embedding API 调用慢, 可以检查 API 的调用日志, 看是不是响应慢或者超时重试了。

3.5 异常处理

整个处理流程用 try-catch 包裹, 任何环节出错都会被捕获:

```
try {
    // 上面的处理流程
} catch (Exception e) {
```



```
log.error("文档分块任务执行失败: docId={}", docId, e);
markChunkFailed(documentDO.getId());
long totalDuration = System.currentTimeMillis() - totalStartTime;
updateChunkLog(chunkLog.getId(), DocumentStatus.FAILED.getCode(), 0,
    extractDuration, chunkDuration, embedDuration, persistDuration, totalDuration, e.getMe
}
```

异常处理做了两件事：

1. **标记文档失败**：调用 `markChunkFailed` 方法，把文档状态改为 `failed`。这个方法会开启一个新事务，确保状态更新成功（即使前面的事务回滚了）。
2. **更新日志记录**：记录失败状态和错误信息。`errorMessage` 字段存储异常的 `message`，方便排查问题。

注意 `markChunkFailed` 方法是在一个新事务中执行的，这是一个常见的异常处理模式。如果在当前事务中更新状态，事务回滚时状态更新也会回滚，文档状态还是 `running`，用户无法重新触发分块。开启新事务可以确保状态更新成功。

原子性写入设计

`persistChunksAndVectorsAtomically` 方法是整个分块流程中最关键的一步，它保证了 `chunk` 表和 `vector` 表的数据一致性。来看完整的实现。

1. 方法实现

```
private int persistChunksAndVectorsAtomically(String collectionName, String docId,
    List<VectorChunk> chunkResults) {
    List<KnowledgeChunkCreateRequest> chunks = chunkResults.stream()
        .map(vc -> {
            KnowledgeChunkCreateRequest req = new KnowledgeChunkCreateRequest();
            req.setChunkId(vc.getChunkId());
            req.setIndex(vc.getIndex());
            req.setContent(vc.getContent());
            return req;
        })
        .toList();

    new TransactionTemplate(transactionManager).executeWithoutResult(status -> {
        knowledgeChunkService.deleteByDocId(docId); // 1. DELETE 旧 chunks
        knowledgeChunkService.batchCreate(docId, chunks); // 2. INSERT 新 chunks
        vectorStoreService.deleteDocumentVectors(collectionName, docId); // 3. DELETE 旧 vectors
        vectorStoreService.indexDocumentChunks(collectionName, docId, chunkResults); // 4. INSERT
        KnowledgeDocumentDO updateDocumentDO = KnowledgeDocumentDO.builder()
            .id(docId)
            .chunkCount(chunks.size())
            .status(DocumentStatus.SUCCESS.getCode())
            .updatedBy(UserContext.getUsername())
            .build();
        documentMapper.updateById(updateDocumentDO); // 5. UPDATE 文档状态
    });
    return chunks.size();
}
```

方法用 `TransactionTemplate` 手动开启事务，在事务中执行五个操作：

1. **DELETE 旧 chunks**：删除文档的所有旧 `chunk` 记录
2. **INSERT 新 chunks**：批量插入新 `chunk` 记录
3. **DELETE 旧 vectors**：删除文档的所有旧向量
4. **INSERT 新 vectors**：批量插入新向量
5. **UPDATE 文档状态**：更新文档状态为 `success`，更新 `chunk` 数量

这五个操作在同一个事务中，要么全部成功，要么全部失败。

2. 为什么要先删后插

你可能会问：为什么不用 UPDATE 更新 chunk，而要先删后插？

假设旧文档有 100 个 chunk，新文档只有 50 个 chunk。如果用 UPDATE：

前 50 个 chunk 可以更新

后 50 个旧 chunk 怎么办？UPDATE 语句无法删除它们

如果用 DELETE + INSERT：

先删除所有旧 chunk（100 个）

再插入所有新 chunk（50 个）

数据完全替换，不会留下脏数据

先删后插是一种简单可靠的数据替换策略，不需要考虑新旧数据的数量差异。

3. 为什么 chunk 表和 vector 表要一起操作

chunk 表和 vector 表必须保持一致。如果 chunk 表插入成功但 vector 表插入失败，会出现什么问题？

用户在前端看到文档有 50 个 chunk

但检索时只能检索到部分 chunk（vector 表只插入了一部分）

用户会觉得系统有 bug，明明有 chunk 但检索不到

反过来，如果 vector 表插入成功但 chunk 表插入失败：

检索能检索到结果

但前端查看 chunk 列表时看不到 chunk 记录

同样会让用户困惑

所以这两个表必须在同一个事务中操作，要么都成功，要么都失败。失败了用户可以重新触发分块，不会出现数据不一致的情况。

4. 向量库的事务支持

你可能会问：向量库（pgvector）支持事务吗？

项目中的 vectorStoreService 有两种实现：

PgVectorStoreService：基于 PostgreSQL + pgvector 扩展。pgvector 的向量表就是普通的 PostgreSQL 表，完全支持事务。
deleteDocumentVectors 和 indexDocumentChunks 方法内部用的是 JdbcTemplate，会自动参与到外层的 Spring 事务中。

MilvusVectorStoreService：基于 Milvus 向量数据库。Milvus 不支持事务，deleteDocumentVectors 和 indexDocumentChunks 是独立的操作，无法和 PostgreSQL 的事务保持一致。

项目默认用的是 PgVectorStoreService，所以 persistChunksAndVectorsAtomically 方法可以保证原子性。如果切换到 Milvus，就无法保证原子性了，需要用其他方案（比如先写 Milvus，成功后再写 PostgreSQL，失败时通过补偿机制清理 Milvus 中的数据）。

这也是为什么项目选择 pgvector 而不是 Milvus 的原因之一：pgvector 和 PostgreSQL 在同一个数据库里，事务一致性更好，运维更简单。

5. 事务边界设计

回过头看整个流程，会发现事务边界的设计很精细：

startChunk 方法：没有加 @Transactional 注解。本地事务由 RocketMQ 事务消息框架管理——sendInTransaction 内部会用 TransactionTemplate 包裹回调中的数据库操作，保证 CAS 状态更新和消息投递的原子性。事务很短，几毫秒就完成了。

runChunkTask 方法：没有加 @Transactional 注解，因为它包含耗时的文件解析、分块、向量化操作。如果加了事务，这些操作都会在事务中执行，事务会持有数据库连接几分钟，导致连接池耗尽。

persistChunksAndVectorsAtomically 方法：内部用 TransactionTemplate 手动开启事务，只包含数据库和向量库的写入操作。事务边界精确控制，只在需要原子性的地方开启事务。

这种设计遵循了一个原则：**事务内不做耗时操作**。耗时操作（文件 IO、网络请求、复杂计算）都在事务外执行，只有数据库写入操作在事务内执行。这样可以避免长事务导致的连接池耗尽、锁等待等问题。

分块日志的作用

前面提到了分块日志表 `t_knowledge_document_chunk_log`，这个表在实际运行中非常有用。来看它的字段设计和使用场景。

1. 字段设计

```
@Data
@TableName("t_knowledge_document_chunk_log")
public class KnowledgeDocumentChunkLogDO {
    private String id;
    private String docId;
    private String status;           // running / success / failed
    private String processMode;     // chunk / pipeline
    private String chunkStrategy;   // 分块策略 (CHUNK 模式)
    private String pipelineId;      // Pipeline ID (PIPELINE 模式)
    private Long extractDuration;   // 文本提取耗时 (毫秒)
    private Long chunkDuration;    // 分块耗时 (毫秒)
    private Long embedDuration;    // 嵌入 API 调用耗时 (毫秒)
    private Long persistDuration;  // DB 持久化耗时 (毫秒)
    private Long totalDuration;    // 总耗时 (毫秒)
    private Integer chunkCount;    // 生成的 chunk 数量
    private String errorMessage;   // 错误信息 (失败时)
    private Date startTime;
    private Date endTime;
}
```

每次执行分块任务都会创建一条新记录，记录完整的执行情况。
前端示例如下，直接分块的截图：

分块详情

文档 [SKILL_NAME_CONFLICT_FIX.md] 的分块执行日志

成功

直接分块 · 固定大小

1 块

文本提取

239ms

分块耗时

3ms

向量化

1.01s

持久化

157ms

其他

2ms

总耗时

1.41s

执行时间 2026/3/29 11:48:35 ~ 2026/3/29 11:48:37

关闭

2. 性能分析

通过各阶段耗时指标，可以定位性能瓶颈：

- extractDuration 很长**：说明文件解析慢。可能是 PDF 格式复杂、文件很大、或者 Tika 解析器性能问题。
- chunkDuration 很长**：说明分块慢。可能是文本量大、分块策略复杂。
- embedDuration 很长**：说明 Embedding API 调用慢。可能是 chunk 数量太多、Embedding API 响应慢、或者网络问题。
- persistDuration 很长**：说明数据库和向量库写入慢。可能是向量库性能问题、批量插入的 batch size 太小、或者数据库负载高。

比如发现某个文档的 `embedDuration` 是 120 秒，而 `extractDuration` 只有 2 秒，说明瓶颈在 Embedding 环节。可以检查 Embedding API 的调用日志，看是不是 API 响应慢或者超时重试了。

3. 问题排查

失败时 `errorMessage` 字段记录了异常信息，方便排查问题：

"Embedding API timeout": Embedding API 超时，可能是网络问题或者 API 服务不稳定

"Failed to parse PDF": PDF 解析失败, 可能是文件损坏或者格式不支持

"Vector store connection failed": 向量库连接失败, 可能是向量库宕机或者网络问题

有了这些信息, 可以快速定位问题, 而不需要翻遍所有日志。

4. 历史记录

文档可能会多次重新分块 (比如调整分块参数后重新处理), 每次执行的情况都不一样。分块日志表保留了完整的历史记录, 可以对比不同参数下的性能表现。

比如想知道 `chunkSize=500` 和 `chunkSize=1000` 哪个更快, 可以查询日志表, 对比两次执行的 `totalDuration`。

FAQ

1. 为什么不在 upload 接口中自动触发分块

最直接的原因是耗时差异。upload 接口耗时短 (几秒), 用户可以等。分块耗时长 (几分钟), 用户等不了。

但更深层的原因是职责分离。upload 接口的职责是存文件, startChunk 接口的职责是分块。两个接口做两件事, 职责清晰, 代码更好维护。

拆开之后还有这些好处:

灵活控制: 用户可以上传多个文档后批量触发分块, 也可以文档更新后只重新分块不重新上传。

支持重试: 分块失败后, 用户可以直接重新触发分块, 不需要重新上传文件。

方便扩展: 后续如果要支持分块参数调整 (比如改变 chunkSize), 只需要重新触发分块, 不需要重新上传文件。

2. 为什么用 CAS 更新状态而不是分布式锁

你可能会问: 为什么不用分布式锁 (比如 Redisson) 来防止重复触发?

关键在于: 这个场景根本不需要锁。

场景不需要锁: startChunk 要解决的问题是防重复触发, 本质是状态判断 + 状态更新。CAS 用一条 SQL 就搞定了, WHERE status != 'running' 天然原子性, 不需要额外引入锁机制。

语义更匹配: CAS 表达的是只有状态符合预期才更新, 这正好就是防重复触发的语义。分布式锁表达的是我要独占这个资源一段时间, 但 startChunk 不需要独占任何资源, 它只是改个状态、发个消息就结束了。

架构更简洁: CAS 只依赖数据库, 不额外依赖 Redis。虽然项目里有 Redis, 但能少一个依赖点就少一个故障点。用分布式锁意味着锁的获取和释放都要走 Redis, 多了一次网络往返, 而 CAS 在数据库层面一步到位。

打个比方: 你要判断一张桌子有没有人坐, 看一眼就知道了 (CAS), 不需要先拿一把锁把整个房间锁起来再去看 (分布式锁)。

3. 为什么 startChunk 方法不加 @Transactional

startChunk 方法的本地事务由 RocketMQ 事务消息框架管理。sendInTransaction 内部的 DelegatingTransactionListener 会用 TransactionTemplate 包裹回调中的数据库操作, 不需要在方法上加 @Transactional。

如果加了 @Transactional 会怎样? 外层事务会包含整个方法的执行, 包括发送半消息的网络请求。万一网络请求耗时较长, 事务就会持有数据库连接等待, 不够精细。

runChunkTask 方法更不能加事务, 它包含耗时的文件解析、分块、向量化操作, 可能要几分钟。如果加事务, 事务会持有数据库连接几分钟, 导致连接池耗尽。

正确的做法是: 耗时操作在事务外执行, 只有数据库写入操作在事务内执行。persistChunksAndVectorsAtomically 方法内部用 TransactionTemplate 手动开启事务, 只包含数据库和向量库的写入操作, 事务边界精确控制。

4. 如果消息发送或消费失败怎么办

分两种情况:

发送阶段失败: RocketMQ 事务消息保证了本地事务和消息发送的原子性。如果半消息发送失败, 本地事务不会执行; 如果本地事务执行失败, 消息会被回滚。如果应用在通知 Broker 之前宕机, Broker 会通过事务回查机制查询本地事务状态, 决定提交还是回滚。所以发送阶段不会出现数据不一致。

消费阶段失败: RocketMQ 支持消费重试机制。消费者处理失败 (抛出异常), 消息会被重新投递。如果多次重试都失败, 消息会进入死信队列, 需要人工介入排查。

重试机制可以应对临时故障 (比如网络抖动、API 超时), 但无法应对永久故障 (比如文件损坏、格式不支持)。永久故障需要人工介入, 修复问题后重新触发分块。

5. 为什么 persistChunksAndVectorsAtomically 要先删后插

如果用 UPDATE 更新 chunk, 会遇到数量不匹配的问题。

假设旧文档有 100 个 chunk，新文档只有 50 个 chunk：

- 前 50 个 chunk 可以 UPDATE
- 后 50 个旧 chunk 怎么办？UPDATE 语句无法删除它们
- 如果不删除，数据库里会留下 50 个脏数据

先删后插可以避免这个问题：

- 先删除所有旧 chunk（100 个）
- 再插入所有新 chunk（50 个）
- 数据完全替换，不会留下脏数据

这是一种简单可靠的数据替换策略，不需要考虑新旧数据的数量差异。

小结

回过头看，startChunk 接口的设计体现了几个关键思想：

- 职责分离**：upload 只管存文件，startChunk 只管分块。两个接口做两件事，职责清晰。
- 异步处理**：分块耗时长，必须异步。用 RocketMQ 事务消息解耦，保证本地事务和消息投递的原子性，支持失败重试和削峰填谷。
- 并发控制**：用 CAS 更新状态，防止重复触发。轻量、高效、不会死锁。
- 原子性保证**：chunk 表和 vector 表在同一个事务中操作，要么都成功，要么都失败。
- 事务边界精确控制**：耗时操作在事务外执行，只有数据库写入在事务内执行。避免长事务导致的连接池耗尽。
- 可观测性**：分块日志记录完整的执行情况，方便性能分析和问题排查。

至于具体的分块代码实现（文档解析、各种分块策略的代码细节），这部分相对固定，不是系统设计的重点。前面 RAG 基础篇已经讲过分块是什么、有哪些分块方式，知道这些就够了。感兴趣的同学可以直接看源码，后续如果有需要也可以补充专门的文章。
下一篇文章我们讲知识库的定时同步机制：系统怎么按照用户配置的 cron 表达式，定时抓取远程 URL 文档并重新分块，以及定时任务的分布式调度设计。